Algoritmos

Los Palmeras

2026-04-22

```
## Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código.

El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Mark

## Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.



```r
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```r
# Planteamos el dataset
data(cars)
```

```r
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

```r
summary(cars)
```

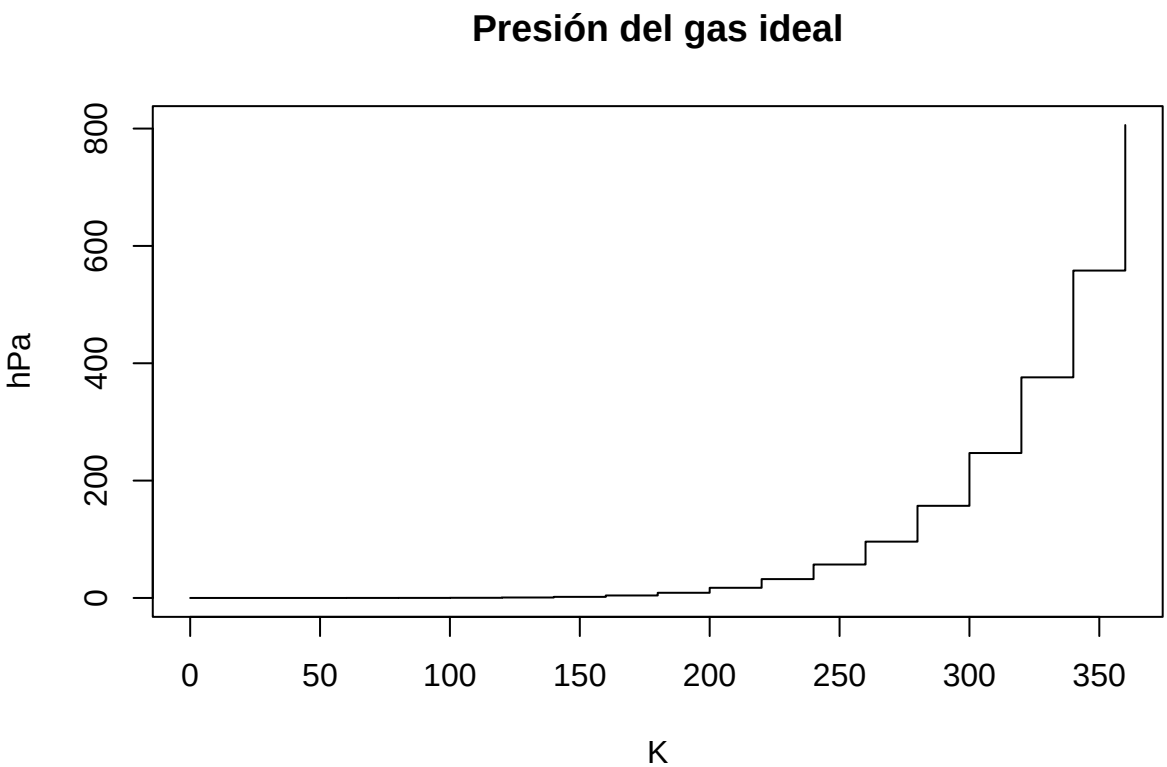
```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
```
```

```
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```r
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```r
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 31.268162 15.866934 26.029462 27.389174 15.538746 28.805326 25.503639
##   [8] 23.868508 25.618240 15.611591 27.052561 27.952001 25.139640 21.368484
##  [15] 33.570664 25.610338 23.513699 17.755510 22.877727 17.366429 20.545984
```

```
##  [22] 20.935486 17.470541 23.797461 19.262575 22.275796 24.278810 21.728339
##  [29] 17.644978 21.361998 10.357215 22.578988 22.115239 27.178709 18.198033
##  [36] 14.156531 20.426382 23.026394  9.814219 14.730166 25.449850 26.273824
##  [43] 34.121460 25.637956 21.475291 11.332338 23.990612 21.649303 15.881550
##  [50] 24.172432 22.803199 11.892696 24.403492 26.717056 29.323181 32.390426
##  [57] 23.333016 26.840439 14.990729 17.123317 22.989866 26.331733 24.176752
##  [64] 18.885154 19.829294 25.878750 23.726216 20.559737 19.853760 19.556001
##  [71] 26.628186 19.176783 29.012056 20.911376 26.284084 24.686307 17.579692
##  [78] 22.757646 26.277346 24.586223 29.707579 18.601940 22.055151 21.846932
##  [85] 21.383517 21.624888 14.436405 19.605534 19.652540 18.500539 20.428217
##  [92] 17.014361 17.208172 25.484043 19.558485 21.575704 31.211797 18.841646
##  [99] 21.401172 20.760058 22.479586 20.368153 23.343309 26.068335 19.106176
## [106] 16.510569 18.131697 20.881629 16.669034 24.744371 25.367246 25.588776
## [113] 20.224088 14.375239 28.198534 20.598384 25.990579 15.018182 22.543119
## [120] 18.294350 18.149153 10.078380 17.523107 23.314789 32.668533 26.404971
## [127] 13.308310 24.807977 27.008305 20.271003 27.199112 24.711009 27.359722
## [134] 21.631405 22.999862 25.960153 21.504919 21.342639 22.412034 15.780224
## [141] 16.769965 22.124112 19.830243 15.772637 16.742162 19.315675 28.653617
## [148] 26.752744 24.353016 28.906725 20.825775 15.411377 26.154177 23.163690
## [155] 17.893250 24.997068 22.543923 21.862796 24.099827 21.523070 21.273322
## [162] 18.673541 20.711922 11.968029 18.192746 23.799360 20.309925  8.235159
## [169] 28.486170 22.523563 28.201099 26.043247 32.433869 18.448876 14.153713
## [176] 23.797325 28.758374 27.373466 26.247468 23.251201 24.013556 19.192143
## [183] 17.697509 24.762588 22.948693 27.461977 22.680041 26.324727 22.187911
## [190] 23.475505 19.194387 21.140584 15.217031 18.576411 18.154163 24.975491
## [197] 17.988298 19.856261 26.311938 24.757982 29.254900 34.235254 24.931571
## [204] 29.940271 18.768228 25.703483 17.799807 23.508537 23.625561 22.541857
## [211] 20.663937 24.216080 22.183023 28.853297 15.573167 22.889519 15.846990
## [218] 18.436470 19.434670 24.761230 25.381260 23.120443 22.530104 19.656452
## [225] 22.568627 23.055364 24.747458  9.129939 21.895291 29.313744 17.965382
## [232] 20.867942 22.616640 14.750348 12.750136 24.630027 27.117616 25.473335
## [239] 23.280721 29.025677 21.195757 18.024738 21.992883 20.298353 13.473099
## [246] 24.197293 20.778297 27.990610 11.858111 28.864519 15.183447 19.044598
## [253] 16.982976 18.762550 19.735230 23.718358 19.228922 24.146423 26.271788
## [260] 24.969908 21.453864 19.601331 31.086267 25.856994 14.545448 28.126663
## [267] 18.438676 36.775041 33.393152 24.002709 19.463255 17.806818 24.854033
## [274] 26.868258 27.370944 22.818550 29.346451 25.291329 23.277525 18.271362
## [281] 28.402860 27.763398 11.899747 19.063625 19.967206  9.787231 24.146346
## [288] 26.050674 24.114544 25.054619 20.896818 25.398439 12.273782 26.536348
## [295] 21.901689 23.812417 20.618110 15.133575 28.548272 31.268516 23.053539
## [302] 18.387052 25.417608 20.961777 29.517465 23.145273 20.705563 31.459506
## [309] 19.448727 15.697103 15.447524 23.456888 19.438807 30.581384 17.839051
## [316] 22.855867 18.164852 27.065367 16.575928 22.236499 20.944348 19.060913
## [323] 22.397640 31.076979 24.336745 18.469155 21.159099 18.080387 22.477948
## [330] 21.684580 28.100444 24.612160 19.824282 23.853358 29.260217 24.796100
## [337] 16.627620 14.839599 31.151746 22.195377 22.402385 24.519396 15.012753
## [344] 22.518274 27.055425 26.233838 21.020078 34.120376 16.748223 16.269692
```

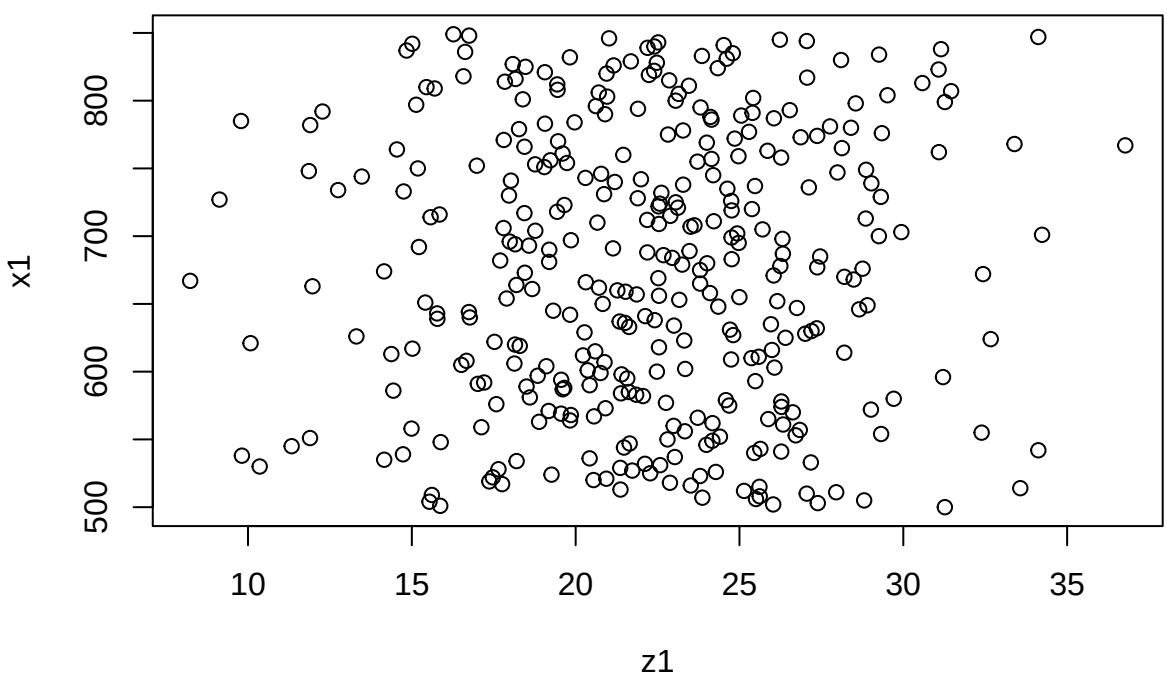
```r
w1 <- length(z1)
w1
```

```
## [1] 350
```

```r
x1 <- 500:849
x1
```

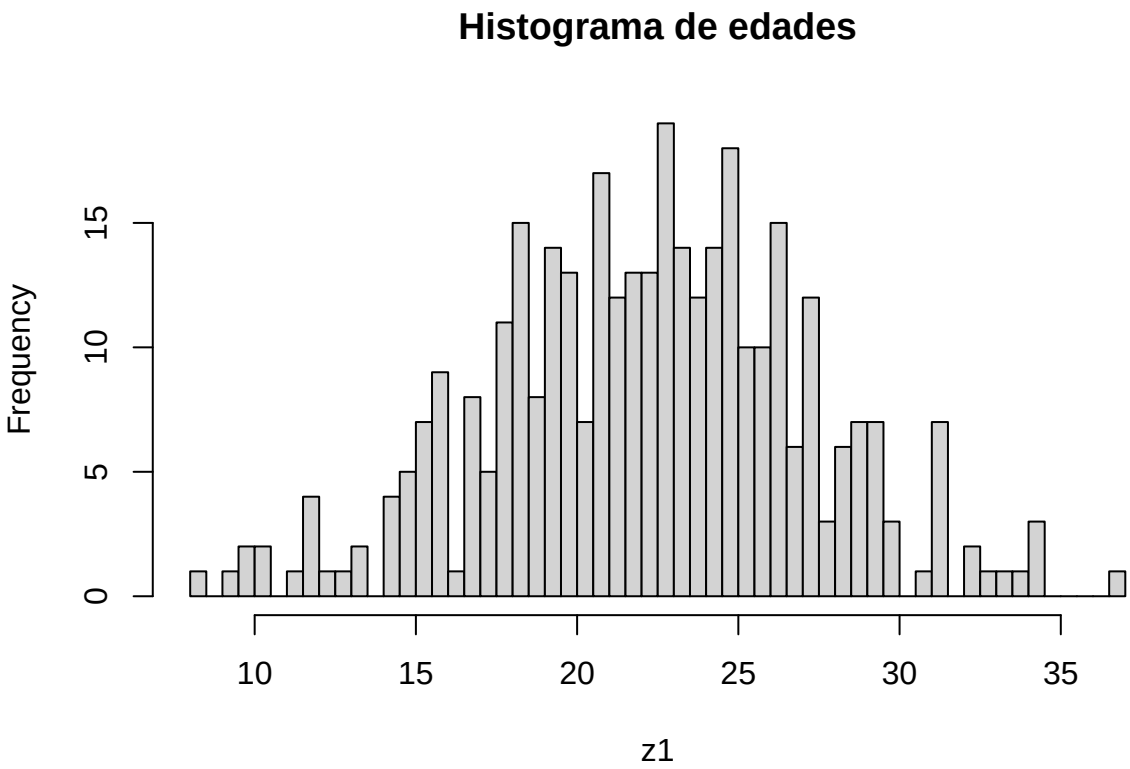
```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



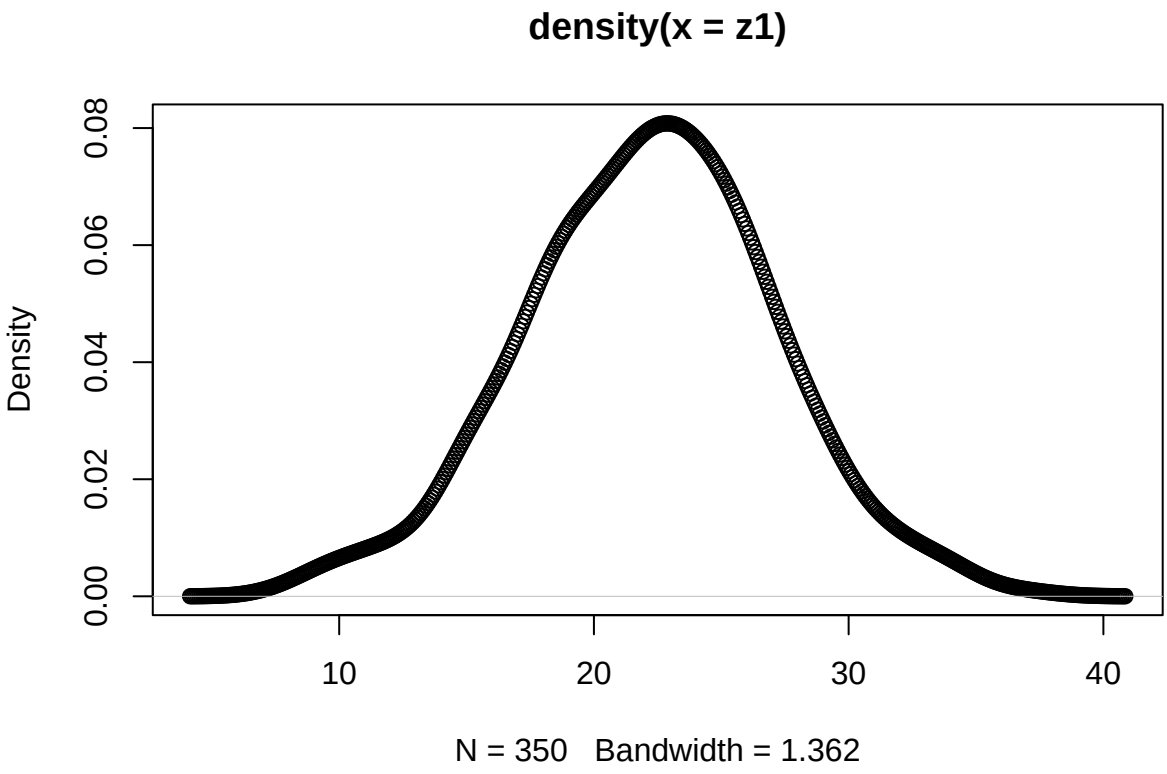
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.362
##
##        x               y
##  Min.   : 4.15   Min.   :9.480e-06
##  1st Qu.:13.33   1st Qu.:2.727e-03
##  Median :22.51   Median :1.337e-02
##  Mean   :22.51   Mean   :2.721e-02
##  3rd Qu.:31.68   3rd Qu.:5.208e-02
##  Max.   :40.86   Max.   :8.081e-02
```

```
plot(density(z1),type="b")
```



```
## Ejercicio 1: Designación
```

Consigna: Las ultimas 3 cifras del DNI son 364, crear una variable que tenga ese número

```
DNI <- 364
```

Consigna: Crear un vector del 1 al 364.

```
lista_dni <- 1:364
lista_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364
```

```
## Ejercicio 2: Uso de for
```

Consigna: Calcular la suma de todos los valores del vector *lista_dni* usando un for:

```r
Total <- 0
valor_final <- length(lista_dni)

for (i in 1:valor_final) {
Total <- Total + i
}

Total
```

```
## [1] 66430
```

```
## Ejercicio 3: Python

**Consigna:** Repetir el ejercicio anterior pero en Phyton
```

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (364)
dni = 364

# Consigna: Crear un vector (lista) del 1 al 364
# En Python, range(1, 364) genera números del 1 al 364
lista_dni = list(range(1, dni + 1))

# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in lista_dni:
    total += i

# Mostrar el resultado final
print(f"Suma total: {total}")
```

```
## Suma total: 66430
```

```
## Ejercicio 4: Medición de tiempo (sys.time)

**Consigna:**¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**
```

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(lista_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 6.6248e+18
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 59.15601 secs
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(lista_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.

#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.005 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
```

Total
```

```
## [1] 66430
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
    A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.006387234 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.002194166 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000
```
```

```r
fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
    nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
    fibonacci <- c(fibonacci, nuevo)
    iteraciones <- iteraciones + 1
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```r
iteraciones
```

```
## [1] 32
```

```
## Ejercicio 8: Definición matemática recurrente
```

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```r
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

f2=f0+f1
f0=f1
f1=f2
i<-i+1

}
f2
```

```
## [1] 1346269
```

```r
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```r
x <- sample(1:20000, 20000)

burbuja <- function(x) {
    n <- length(x)
    for (j in 1:(n-1)) {
        for (i in 1:(n-j)) {
            if (x[i] > x[i+1]) {
                temp <- x[i]
                x[i] <- x[i+1]
                x[i+1] <- temp
            }
        }
    }
    return(x)
}
```

Comparación:

```r
system.time({
res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.789   0.044  21.022
```

```r
system.time({
res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
suma1 <- 0
```

```
for (i in 1:n) {
    suma1 <- suma1 + i
}
})
```

```
##    user  system elapsed
##   0.018   0.000   0.017
```

Método 2 (fórmula matemática):

```r
system.time({
suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

```
## [1] 500000500000
```

```r
suma2
```

```
## [1] 500000500000
```

```
## Conclusión final:
```

En este trabajo se evaluó la eficiencia de diversos algoritmos en R utilizando técnicas de benchmarking (Sys.time, tictoc y microbenchmark). Se comprobó empíricamente que la calidad de un código no radica únicamente en su exactitud, sino en la optimización de los recursos de procesamiento y el tiempo de ejecución. El ejercicio "La penitencia de Newton" ilustra claramente esta premisa: mientras que un bucle iterativo presenta un alto costo computacional ante operaciones masivas, la aplicación de un modelo matemático adecuado resuelve el problema de forma casi instantánea. Las métricas obtenidas confirman que el razonamiento lógico-matemático previo supera a la capacidad del hardware como principal factor de optimización. Los métodos analíticos resultan órdenes de magnitud más eficientes que la iteración tradicional.